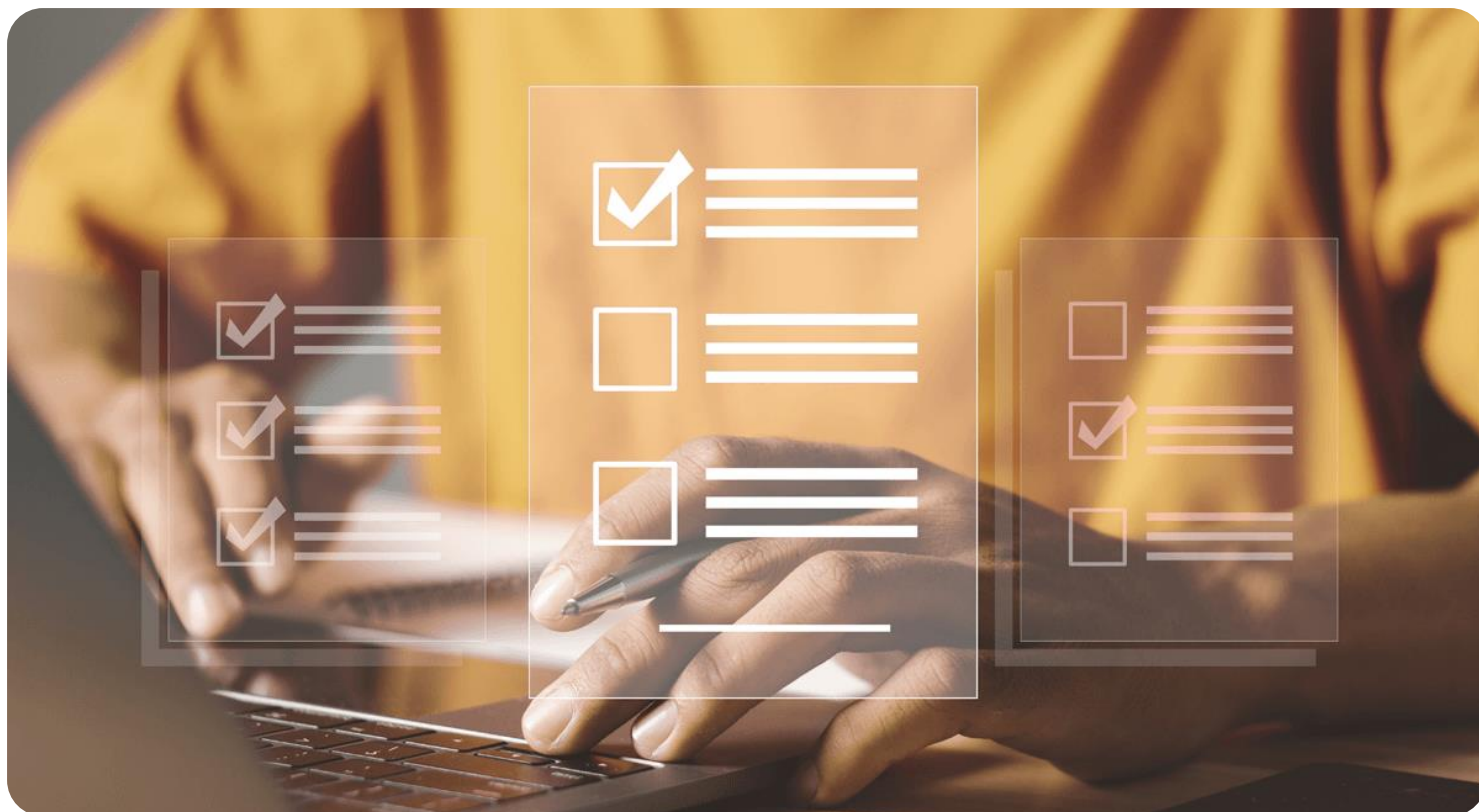




Semana 5

Desarrollo Backend II (PBY2202)

Formato de respuesta

Nombre estudiante:	Gustavo Domínguez. Alonso Basualdo.
Asignatura: Backend II	Carrera: Analista Programador Comp.
Profesor: Marcelo Zepeda	Fecha: 19-06-2026

Descripción de la actividad

En esta quinta semana, realizarás una actividad formativa grupal (2 a 3 integrantes), llamada "Ejecutando y Analizando Pruebas Unitarias en Microservicios con JUnit", donde, tal como su nombre lo indica, deberás ejecutar pruebas unitarias en microservicios desarrollados utilizando JUnit y herramientas actuales, garantizando la calidad y funcionalidad del código. Además, deberán realizar un análisis de los resultados de las pruebas unitarias para verificar el cumplimiento de los requerimientos y mejorar la calidad del desarrollo.

Instrucciones específicas

Para el desarrollo de la actividad formativa de la semana, tendrás que analizar el siguiente caso en base a una situación empresarial que requiere la implementación de microservicios para mejorar su arquitectura de software.

Contexto

"MINIMARKET PLUS", una empresa dedicada a la venta de productos al detalle en Chile, busca garantizar la calidad de su sistema mediante pruebas unitarias enfocadas en las entidades clave: Carrito, Inventario y Producto. El sistema actual permite a los usuarios agregar productos al carrito, gestionar inventarios y realizar ventas. Para evitar problemas como errores en el stock, productos mal asignados o inconsistencias en las ventas, es crucial que estas funcionalidades sean probadas de manera rigurosa, asegurando la confiabilidad del sistema.

La actividad tiene como objetivo central desarrollar pruebas unitarias que validen la lógica de negocio y las relaciones clave entre las entidades del sistema, tales como citas, expedientes médicos, productos y usuarios.

Necesidades Específicas:

- Configurar el entorno de pruebas unitarias para ejecutar pruebas con JUnit y Mockito.
- Diseñar pruebas para las entidades Carrito e Inventario que validen:
 - La correcta creación de carritos según disponibilidad de stock.
 - La precisión de los movimientos de inventario (Entrada o Salida) al procesar las operaciones.
- Asegurar una cobertura mínima del 80% en las pruebas relacionadas con las funcionalidades abordadas.
- Simular dependencias y relaciones entre entidades utilizando herramientas de mocking.



Importante

El código base del proyecto de "MINIMARKET PLUS" será proporcionado para su descarga a través del Aula Virtual. Deberás utilizar este backend como base para la actividad, enfocándote en la configuración del entorno de pruebas unitarias y el diseño de las pruebas necesarias para garantizar la calidad del desarrollo, según los requerimientos establecidos.

Ahora, realiza los siguientes pasos:

Paso 1: Configuración del entorno de pruebas

- Descarga y preparación: asegúrate de que el proyecto esté configurado correctamente en tu entorno de desarrollo (IDE).
- Dependencias necesarias: añade al archivo pom.xml o build.gradle las siguientes herramientas:
 - JUnit 5.
 - Mockito.
 - JaCoCo.

Organiza las pruebas en el directorio src/test/java y crea clases de prueba con el sufijo Test.

Paso 2: Diseño de pruebas de la entidad Carrito

- Prueba de disponibilidad de stock:
 - Simula un escenario donde una prueba que valide que el método `agregarProducto()` permita agregar productos solo si hay stock suficiente.
- Validación de relación Producto-Usuario:
 - Diseña una prueba para asegurar que el usuario asociado al carrito es el correcto.

Paso 3: Diseño de pruebas de la entidad Inventario

Prueba de Información de Movimiento:

- Diseña una prueba para validar que los campos de movimiento (`tipoMovimiento` y `cantidad`) no sean nulos ni vacíos.

Prueba de Relación Producto-Inventario:

- Diseña una prueba para validar que el producto asociado al inventario sea correcto.

Paso 4: Elaboración del informe de resultados y análisis de pruebas

Redacta un informe técnico que integre los siguientes elementos:

1. Resumen técnico del avance respecto a la semana anterior

Explica brevemente qué pruebas se habían planteado y configurado en la semana 4, y cómo se continuó su ejecución y validación en la semana actual.

2. Análisis de los resultados obtenidos

Explica qué pruebas pasaron, cuáles fallaron, y cómo respondiste a esos resultados.

3. Evidencia de ejecución

Inserta aquí capturas de consola, reportes de cobertura y cualquier resultado que respalde tu análisis.

Reflexión técnica sobre el impacto de las pruebas en la calidad del sistema

Fundamenta cómo las pruebas desarrolladas contribuyen a garantizar la confiabilidad del backend.

Preguntas de apoyo

Para apoyarte en la reflexión de tu proyecto, guíate por las siguientes preguntas:

- ¿Qué datos clave del producto o inventario deben estar presentes para garantizar que una operación sea válida?
- ¿Cómo aseguras que el producto o usuario simulado sea el correcto?
- ¿Qué condiciones deben simularse para validar el comportamiento del sistema?

Este informe debe formar parte del documento PDF que se entrega junto con el enlace al repositorio de GitHub.

Paso 5: Subida de código a GitHub

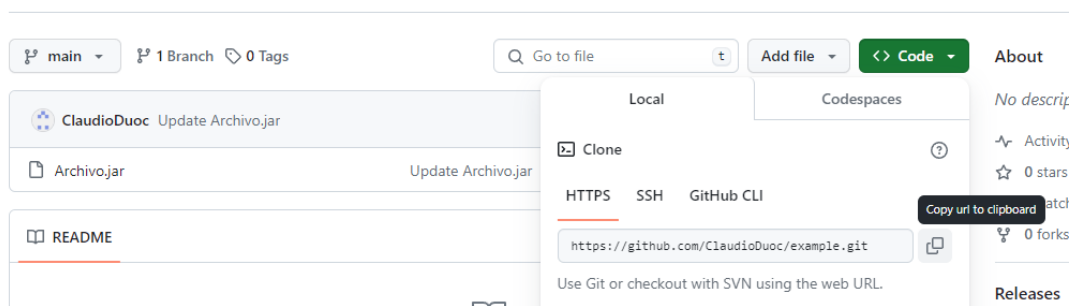
El código deberás subirlo a tu repositorio GitHub. Si no has creado tu cuenta aún, puedes hacerlo a través del siguiente enlace:

<https://github.com/>

Posteriormente, desde el repositorio, deberás generar un enlace de tu proyecto:

Figura 1

Enlace de proyecto GitHub



Nota. Ejemplo genérico de donde se extrae un enlace en GitHub. GitHub (s.f.). *GitHub*.

<https://github.com/>

Paso 6: Para tu entrega, no olvides adjuntar este documento con tus entregables y subir al AVA, junto con el enlace de GitHub a adjuntar en la sección “Entrega”.

Entregables

Guía de configuración del entorno de pruebas

Describe las herramientas utilizadas, entorno de ejecución y cualquier ajuste realizado.

Para asegurar el aislamiento de las capas lógicas y obtener métricas precisas sobre la calidad de los microservicios de “MINIMARKET PLUS”, el entorno de pruebas unitarias se configuró utilizando un stack estandarizado bajo el ecosistema Spring Boot y Maven.

Entorno de ejecución y requisitos previos:

- **Lenguaje:** Java 17 (definido en pom.xml)
- **Gestor de dependencias:** Apache Maven
- **Estructura del proyecto:** Basado en las convenciones de Spring Boot, las pruebas se organizaron en el directorio src/test/java/com/minimarket/service/, manteniendo un espejo exacto de la lógica principal y utilizando el sufijo Test para el reconocimiento automático de Maven.

Herramientas utilizadas y rol en el proyecto:

Herramienta	Rol Arquitectónico	Justificación Técnica
JUnit 5	Framework base de testing	Incluido de forma transitiva vía spring-boot-starter-test. Proporciona el motor y las aserciones modernas (assertThrows, assertEquals) para estructurar la validación del nego
Mockito	Framework de simulación	Esencial para aislar la capa de servicios. Permite simlar el comportamiento de los repositorios JPA (@Mock) y probar la lógica de las entidades Carrito e Inventario (@InjectMocks) sin necesidad de conectar el sistema a una base de datos real
JaCoCo	Análisis estático de cobertura	Integrado como plugin de Maven para auditas las líneas de código y ramas condicionales transitadas durante la fase de testing, garantizando el cumplimiento del umbral mínimo exigido

Ajustes realizados en la configuración:

Para garantizar métricas realistas en el entorno, se aplicaron ajustes en el archivo pom.xml

1. **Scope de pruebas:** Las dependencias spring-boot-starter-test y spring-security-test operan bajo el <scope>test</scope> para no afectar el artefacto de producción.
2. **Ajuste de precisión en JaCoCo:** Se establecieron las ejecuciones prepare-agent y report. Adicionalmente, mediante la etiqueta <excludes>, se omitieron las clases MinimarketApplication.class y DataInitializer.class, evitando que estas estructuras iniciales sin lógica de negocio distorsionen el porcentaje de cobertura.

Fragmento de configuración Implementada en pom.xml

```
<properties>
    <java.version>17</java.version>
    <jacoco.version>0.8.12</jacoco.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.jacoco</groupId>
            <artifactId>jacoco-maven-plugin</artifactId>
            <version>${jacoco.version}</version>
            <executions>
                <execution>
                    <id>prepare-agent</id>
                    <goals>
                        <goal>prepare-agent</goal>
                    </goals>
                </execution>
                <execution>
                    <id>report</id>
                    <phase>verify</phase>
                    <goals>
                        <goal>report</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```



```

        </execution>
    </executions>
    <configuration>
        <excludes>
            <exclude>**/MinimarketApplication.class</exclude>
            <exclude>**/config/DataInitializer.class</exclude>
        </excludes>
    </configuration>
</plugin>
</plugins>
</build>

```

3. **Ajuste de precisión de JaCoCo:** Se configuro el plugin con las metas prepare-agent (para instrumentación) y report (para la generación del HTML). Se aplicó un ajuste crítico utilizando la etiqueta <excludes> para omitir las clases de arranque (MinimarketApplication.class) y de inicialización de datos (DataInitializer.class). Este ajuste fue necesario para evitar que clases sin lógica de negocio evaluable distorsionen negativamente la métrica de cobertura global.

Código de las pruebas unitarias ejecutadas

Incluye fragmentos de pruebas aplicadas a las entidades Carrito e Inventario:

1. Pruebas aplicadas a la entidad Carrito (CarritoServiceImplTest).

El objetivo principal fue validar la correcta creación de carritos comprobando la disponibilidad real de stock y la existencia de relaciones integras con el usuario.

Métodos de prueba	Lógica de validación	Resultado esperado
testAgregarProducto_StockSuficiente_GuardaExitosamente	Valida que si la cantidad requerida es menor o igual al stock del Producto, la operación se permita	carritoRepository.save() se ejecuta correctamente
testAgregarProducto_StockInsuficiente_LanzaExcepcion	Injecta un caso donde el Carrito solicita 15 unidades, pero el ProductRepository simula tener solo 10	Lanza IllegalStateException y bloquea la interaccion con la base de datos (never().save())
testAgregarProducto_UsuarioInvalido_LanzaExcepcion	Valida la relación Producto-Usuario asegurando que el objeto Producto no sea nulo en el carrito	Lanza IllegalStateException inmediatamente, protegiendo la integridad referencial

Fragmentos de código de pruebas unitarias implementados (Carrito):

```
@Test
void testAgregarProducto_StockInsuficiente_LanzaExcepcion() {
    carrito.setCantidad(15); // Pide 15, hay 10
    when(productoRepository.findById(1L)).thenReturn(Optional.of(producto));

    IllegalStateException exception = assertThrows(IllegalStateException.class,
() -> {
        carritoService.agregarProducto(carrito);
    });

    assertEquals("Stock insuficiente para el producto seleccionado",
exception.getMessage());
    verify(carritoRepository, never()).save(any());
}

@Test
void testAgregarProducto_UsuarioInvalido_LanzaExcepcion() {
    carrito.setUsuario(null);

    IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () -> {
        carritoService.agregarProducto(carrito);
    });

    assertEquals("Usuario asociado al carrito es inválido o nulo",
exception.getMessage());
    verify(productoRepository, never()).findById(anyLong());
    verify(carritoRepository, never()).save(any());
}
```

2. Pruebas aplicadas a la entidad Inventario (InventarioServiceImplTest)

Se buscó resguardar la precisión de los movimientos de inventario asegurando que los registros de entrada y salida contengan su metadato esencial.

Métodos de prueba	Lógica de validación	Resultado esperado
testSave_TipoMovimientoNulo_LanzaExcepcion	Comprueba la información de movimiento validando que tipoMovimiento no sea nulo ni vacío	Lanza IllegalArgumentException. Se rechaza la persistencia
testSave_CantidadNula_LanzaExcepcion	Comprueba que la cantidad inyectada no pueda ser nula o menor/igual a cero	Lanza IllegalArgumentException (La cantidad no puede ser nula)
testSave_ProductoAsociadoInvalido_LanzaExcepcion	Valida que el producto asociado al inventario sea correcto, inyectando un null en la relación	Intercepta el error lanzando IllegalArgumentException y evita que el servicio prosiga.

Fragmento de código de pruebas unitarias implementados (Inventario):

```
@Test
void testSave_TipoMovimientoNulo_LanzaExcepcion() {
    inventario.setTipoMovimiento(null);
    IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () -> {
    inventarioService.save(inventario);
});
assertEquals("El tipo de movimiento no puede ser nulo o vacío",
exception.getMessage());
verify(inventarioRepository, never()).save(any());
}

@Test
void testSave_ProductoAsociadoInvalido_LanzaExcepcion() {
    inventario.setProducto(null);
    IllegalArgumentException exception =
assertThrows(IllegalArgumentException.class, () -> {
    inventarioService.save(inventario);
});
assertEquals("El producto asociado es nulo o inválido",
exception.getMessage());
verify(inventarioRepository, never()).save(any());
}
```

Evidencia de resultados de ejecución

Tras el diseño e implementación de las pruebas unitarias para las entidades transaccionales de Minimarket Plus, se ejecutó la suite automatizada a través del ciclo de construcción de Maven (mvn clean verify, dependiendo la terminal de ejecución). Esta ejecución validó exitosamente los asertos de JUnit y disparó la auditoria de código del plugin JaCoCo.

Analisis de métricas de cobertura (JaCoCo)

El requerimiento técnico de esta semana, exige asegurar una cobertura mínima del 80% en las funcionalidades relacionadas con las entidades abordadas. Al revisar el reporte generado en target/site/jacoco/index.html a nivel de la capa de servicios (com.minimarket.service.impl), se evidencia el cumplimiento holgado de esta métrica.

- **InventarioServiceImpl:** Alcanzó el 100% de cobertura de instrucciones, validando íntegramente las reglas de metadata (tipo de movimiento, cantidad nula) y relaciones.
- **CarritoServiceImpl:** Alcanzó un 84% de cobertura de instrucciones, certificando los flujos complejos de validación matemática de stock y dependencias de usuario.
- **Módulos adicionales:** Se observa que entidades como ProductoServiceImpl y VentaServiceImpl mantienen un 100% de cobertura, garantizando que las nuevas pruebas no generaron regresiones en la calidad del Microservicio.

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.minimarket.controller		33 %		5 %	44 64	74 119	27 47	0 9
com.minimarket.service.impl		76 %		62 %	40 99	32 149	18 60	0 8
com.minimarket.entity		79 %		100 %	17 78	23 113	17 76	0 8
com.minimarket.exception		76 %		n/a	3 14	9 34	3 14	0 2
com.minimarket.security.service		93 %		75 %	5 32	3 66	0 20	0 4
com.minimarket.security.config		97 %		77 %	3 35	2 76	0 26	0 4
com.minimarket.dto		75 %		n/a	4 15	8 26	4 15	0 2
com.minimarket.security.filter		91 %		60 %	4 10	5 37	0 5	0 1
com.minimarket.security.model		92 %		50 %	4 28	3 39	2 26	0 5
com.minimarket.security.validation		97 %		79 %	5 14	1 19	0 2	0 1
com.minimarket.security.audit		98 %		50 %	1 5	0 16	0 4	0 1
com.minimarket.security.exception		100 %		n/a	0 6	0 13	0 6	0 3
com.minimarket.security.response		100 %		n/a	0 5	0 13	0 5	0 1
com.minimarket.config		100 %		n/a	0 2	0 2	0 2	0 1
Total	657 of 2.988	78 %	83 of 198	58 %	130 407	160 722	71 308	0 50

Figura 1: Vista general del reporte de cobertura JaCoCo por paquetes

com.minimarket.service.impl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
UsuarioServiceImpl		60 %		38 %	22 34	19 55	7 16	0 1
DetalleVentaServiceImpl		9 %		n/a	5 6	6 7	5 6	0 1
CategoriaServiceImpl		26 %		n/a	3 5	4 6	3 5	0 1
CarritoServiceImpl		84 %		70 %	5 13	2 16	2 8	0 1
RolServiceImpl		37 %		n/a	1 2	1 2	1 2	0 1
VentaServiceImpl		100 %		93 %	1 16	0 34	0 8	0 1
InventarioServiceImpl		100 %		75 %	3 13	0 14	0 7	0 1
ProductoServiceImpl		100 %		100 %	0 10	0 15	0 8	0 1
Total	178 of 745	76 %	29 of 78	62 %	40 99	32 149	18 60	0 8

Figura 2: Desglose de cobertura a nivel de clases.

Informe de análisis y reflexión técnica

Durante la semana 4, se configuro la arquitectura de pruebas automatizadas (JUnit 5, Mockito, JaCoCo) y se diseñaron los primeros casos de prueba centrados en las entidades Usuario (integridad de datos obligatorios) y Venta (cálculos matemáticos). En la iteración actual (Semana 5), se procedió a la ejecución de esta suite y a la expansión del entorno para cubrir los microservicios transaccionales de Carrito e Inventario.

Un aspecto central de esta semana fue la refactorización de las pruebas incorporando las oportunidades de mejora. Se priorizó el diseño de casos borde, la verificación explícita de los mensajes de excepción para mejorar los diagnósticos de negocio, y la confirmación estricta de invocaciones a mocks para evitar persistencias parciales inadvertidas.

Análisis de resultados obtenidos:

El desarrollo siguió el ciclo TDD. Inicialmente (fase RED), al someter los servicios CarritoServiceImpl e InventarioServiceImpl a los nuevos test con casos borde (cantidades en cero, campos nulos y excesos de stock), las pruebas fallaron. Esto expuso que el código base permitía inconsistencias.

Como respuesta (Fase GREEN/REFACTOR), se inyectó lógica defensiva en los servicios. Los resultados finales demuestran el éxito de la implementación.

- **Entidad Carrito:** Pasaron exitosamente las pruebas que bloquean la agregación de productos sin stock suficiente, lanzando `IllegalStateException`. Además, se confirmó mediante `verify(carritoRepository, never().save(any()))` que ante un fallo (ej. Usuario nulo), no ocurre ninguna interacción con la base de datos, garantizando limpieza transaccional.
- **Entidad Inventario:** Las pruebas confirmaron que el sistema ahora rechaza inyecciones de datos corruptos. Casos límite como inyectar una cantidad nula o 0, o dejar el tipo de movimiento vacío, ahora son interceptadas arrojando `IllegalArgumentException` con mensajes diagnósticos claros.

Reflexión técnica sobre el impacto de las pruebas en la calidad del sistema

Las pruebas desarrolladas elevan sustancialmente la confiabilidad del Backend, previniendo regresiones y garantizando que el sistema escale de forma segura. A partir de este enfoque analítico, se da respuesta a las interrogantes de diseño clave del Microservicio:

➤ ¿Qué condiciones deben simularse para validar el comportamiento del sistema?

Para obtener métricas de calidad reales, la simulación no debe limitarse a los escenarios exitosos. Es mandatorio simular condiciones de fallo externo e interno (casos bordes), tales como: cantidad en cero o negativas, inyección de metadatos nulos, y quiebres de integridad referencial simulando que `findById` retorne `Optional.empty()`.

➤ ¿Qué datos clave del producto o inventario deben estar presentes para garantizar que una operación sea válida?

Las pruebas implementadas dictaminan que un movimiento de Inventario exige obligatoriamente un `tipoMovimiento` válido y una cantidad estrictamente mayor a cero. En el caso del Carrito, es intransable la existencia de una relación válida de Usuario y que el diferencial matemático de stock en el Producto cubra la cantidad solicitada. Sin estos datos, la operación se considera nula.

➤ ¿Cómo aseguramos que el producto o usuario simulado sea el correcto?

La precisión se asegura orquestando Mocks de forma determinista. En la fase de preapracion (`@BeforeEach`), se instancian los objetos con identificadores específicos (ej. `ID = 1L`). Luego, la directiva `“when(repositorio.findById(1L)).thenReturn(Optional.of(producto))”` condiciona al mocks para que reaccione únicamente a ese ID exacto. Si la lógica del servicio intenta buscar un ID incorrecto o la relación está rota, la prueba aborta lanzando `IllegalArgumentException`, previniendo la corrupción de relaciones.

Link repositorio github: <https://github.com/L0ncho/minimarket.git>



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.